

Fine Tuning the Environment

...and Making Passes at the Command Line

If you know a few 'idioms', you can control and fine-tune a process' environment.

Since we breathe in the air in our environment, it makes sense for us to improve the quality of that air by using less fossil fuels and planting more trees. Similarly, when Linux processes are born, they start interacting with an inherited environment. There are some pretty cool tools that Linux provides to inspect this environment. By learning a few of these, we can control and fine-tune the process' environment.

Many important variables reside in the environment. `PATH` is one and `LD_LIBRARY_PATH` is another. At times, environment variables affect the execution of a process. They are like control knobs, using which you can change settings. For example, the manual (`man`) for `malloc` mentions the environment variable `MALLOC_CHECK_` which, when set, substitutes another implementation of the allocator, and catches memory corruptions in C code.

Okay, so that's enough of talking—let us get the ball rolling! We know a Linux process is a running program. Creating a process is simple:

```
~/atal> cat > ./tst.sh
sleep 7200
~/atal> chmod +x $f
chmod +x tst.sh
~/atal> ./tst.sh & # start it in background
[1] 32069
~/atal>
```

Let us inspect the environment of this running process we created. Here, the PID of the process created from the shell script is 32069.

```
~/atal> ps eeewww -p 32069
PID TTY STAT TIME COMMAND
```

```
30968 pts/1 S+ 0:00 sh ./tst.sh HOSTNAME=hydg25.in.ibm.com
TERM=xterm SHELL=/bin/bash...
~/atal>
```

Oops, this is quite a listing... But hey, there's no need to rub our eyes—a little scripting can make things pretty palatable.

```
~/atal> ps eeewww -p 32069 | tr '[[space]]' '\n' | grep 'sh'
HOSTNAME=hydg25.in.ibm.com
TERM=xterm
SHELL=/bin/bash
~/atal>
```

Every assignment you see is an environment variable.

If the environment listing is too long, we can always pass it along to *less* and look at it in a *more* leisurely way.

So there's no more guesswork when you suspect some environment variable is not set correctly—just ask `ps` to probe the suspicious process and check out the environment.

Important aside (a.k.a Idiom 1)

Shells also have the following rule: any variable assignment *preceding the command name* turns the variable into an environment variable. So, for example,

```
~/atal> a=1 b=2 c=3 command arg1 arg2 ...
```

...creates three environment variables *a*, *b* and *c* and puts these into the environment of the process created as a result of running a command.

We use the idiom given below to fine tune the environment.